Compositional Skill Routing for LLM Agents: Decompose, Retrieve, and Compose

Anonymous

Abstract

Large language model (LLM) agents increasingly rely on external skills—reusable tool specifications that define capabilities such as code generation, data analysis, or API interaction. While recent work addresses routing queries to a single best-matching skill, real-world tasks often require *composing* multiple skills into coordinated execution plans. We formalize this as the **Compositional Skill Routing** problem: given a complex user query and a large skill library, decompose the query into atomic sub-tasks, retrieve the appropriate skill for each sub-task, and compose an executable plan. We present SKILLWEAVER, a three-stage framework consisting of (1) an LLM-based task decomposer, (2) a bi-encoder skill retriever with FAISS indexing, and (3) a compatibility-aware DAG planner. To support evaluation, we introduce COMPSKILLBENCH, a benchmark of 300 compositional queries over 2,595 curated skills spanning 17 categories, with ground-truth skill chains and difficulty stratification. Our experiments reveal that *task decomposition quality is the primary bottleneck*: oracle decomposition achieves 99.5% skill recall, while the best LLM decomposer reaches only 33.9% at the category level. We further find that metadata-only retrieval matches or outperforms body-aware retrieval, and that decomposer choice significantly impacts downstream routing quality. Our analysis provides actionable insights for building compositional skill routing systems and highlights decomposition as the critical frontier for improvement.

1 Introduction

The agent paradigm for large language models (LLMs) has evolved beyond single-turn generation to encompass tool use, planning, and multi-step task execution (Schick et al., 2023; Qin et al., 2023; Patil et al., 2024). A key architectural pattern emerging in modern LLM agents is the use of *skills*: modular, reusable tool specifications that define specific capabilities along with instructions for when and how to invoke them (Anthropic, 2025). As agent skill libraries grow—with repositories already containing thousands of community-contributed skills—a fundamental routing question arises: *given a user query, which skill(s) should the agent invoke?*

Prior work has largely treated skill routing as a single-skill selection problem, where the goal is to match a query to the single best skill (Xu et al., 2025). However, real-world user queries frequently require *multiple* skills working together. For example, the query "*Download the dataset, transform it, and create visual reports*" requires at least three distinct skills: an API client for downloading, a data processor for transformation, and a visualization tool for report generation.

We formalize this as the **Compositional Skill Routing** problem (Figure 1): given a complex query q and a skill library $\mathcal{S}$, produce an ordered sequence of skills $[s_1, s_2, \ldots, s_k]$ that collectively accomplish the query, where each s_i handles one atomic sub-task. This formulation captures the compositional nature of real-world tasks while maintaining the modularity benefits of skill-based architectures.

We present SKILLWEAVER, a three-stage framework that addresses this problem through:

1. **Decompose**: An LLM-based task decomposer that breaks complex queries into atomic sub-tasks, each requiring exactly one skill.
2. **Retrieve**: A bi-encoder retriever that identifies candidate skills for each sub-task using semantic similarity over skill metadata.
3. **Compose**: A compatibility-aware planner that selects the best skill per step while considering inter-skill compatibility, producing an executable DAG.

To evaluate compositional skill routing, we construct COMPSKILLBENCH, the first dedicated

benchmark for this task. CompSkillBench contains 300 compositional queries over 2,595 curated skills spanning 17 functional categories, with ground-truth skill chains and three difficulty levels. Skills are sourced from 212 public GitHub repositories and deduplicated to ensure quality.

Our experiments yield several key findings:

- **Decomposition is the bottleneck**: With oracle (ground-truth) decomposition, our retriever achieves 99.5% skill Recall@1 and perfect category-level accuracy. LLM decomposition drops this to 33.9% category Recall@1, identifying decomposition as the critical frontier.
- **Metadata suffices for retrieval**: Metadata-only encoding (name + description) matches or outperforms body-aware encoding that includes the full skill specification, suggesting that concise metadata carries the most discriminative signal.
- **Decomposer choice matters**: Qwen2.5-7B significantly outperforms Llama-3.1-8B and Mistral-7B at task decomposition, with the latter two suffering from over-decomposition that degrades downstream retrieval.

2 Related Work

Tool Selection and Routing. Tool selection for LLMs has been studied through API retrieval (Patil et al., 2024; Qin et al., 2023), tool documentation matching (Hao et al., 2024), and hierarchical routing (Xu et al., 2025). SkillRouter (Xu et al., 2025) is closest to our work, introducing a bi-encoder approach for matching queries to individual skills and finding that skill body content is the decisive retrieval signal. However, all these methods select a *single* tool per query. Our work extends skill routing to the compositional setting, where multiple skills must be orchestrated.

Task Decomposition. LLM-based task decomposition has been explored in planning (Huang et al., 2022; Wang et al., 2023), multi-step reasoning (Wei et al., 2022; Zhou et al., 2022), and agentic frameworks (Yao et al., 2023; Shinn et al., 2023). TaskBench (Shen et al., 2023) evaluates tool-augmented LLMs on multi-step tasks but focuses on a fixed set of tools rather than retrieval from a large library. Our decomposition stage builds on these foundations but is specifically designed for the downstream skill retrieval task.

Retrieval-Augmented Generation. Dense retrieval using bi-encoders (Karpukhin et al., 2020; Ni et al., 2022) has been widely applied to knowledge-intensive NLP tasks. We adapt this paradigm for skill retrieval, using semantic embeddings of skill metadata to enable efficient nearest-neighbor search over large skill libraries.

3 Problem Formulation

Skill Library. A skill library $\mathcal{S} = \{s_1, \ldots, s_N\}$ contains N skills. Each skill s_i is a tuple (n_i, d_i, b_i, C_i) where n_i is the name, d_i is a natural language description, b_i is the full specification body (instructions, examples, configuration), and $C_i \subseteq \mathcal{C}$ is a set of functional categories from a taxonomy $\mathcal{C}$.

Compositional Skill Routing. Given a complex query q that requires multiple capabilities, the goal is to produce:

1. A decomposition $D(q) = [t_1, \ldots, t_K]$ of K atomic sub-tasks.
2. A skill assignment $\sigma : [t_1, \ldots, t_K] \to \mathcal{S}^K$ mapping each sub-task to a skill.
3. An execution plan (DAG) $G = (V, E)$ specifying dependencies between steps.

The compositional routing function is $f : q \to (D, \sigma, G)$ optimizing:

$$\max_{D,\sigma,G} \sum_{k=1}^{K} \text{rel}(t_k, \sigma(t_k)) + \lambda \sum_{(i,j)\in E} \text{compat}(\sigma_i, \sigma_j) \tag{1}$$

where $\text{rel}(\cdot)$ measures sub-task–skill relevance and $\text{compat}(\cdot)$ measures inter-skill compatibility.

4 Method: SkillWeaver

SkillWeaver implements compositional skill routing through three cascaded stages (Figure 1).

4.1 Stage 1: Task Decomposition

Given a complex query q, the task decomposer uses an instruction-tuned LLM to produce an ordered list of atomic sub-tasks:

$$D(q) = \text{LLM}(p_{\text{sys}}, p_{\text{user}}(q)) = [t_1, \ldots, t_K] \tag{2}$$

where p_{sys} is a system prompt that instructs the model to decompose queries into sub-tasks, each requiring exactly one skill. The output is constrained to a JSON array of strings for reliable parsing.

We apply the model's native chat template (e.g., ChatML for Qwen models) to ensure proper instruction following. A fallback parser handles

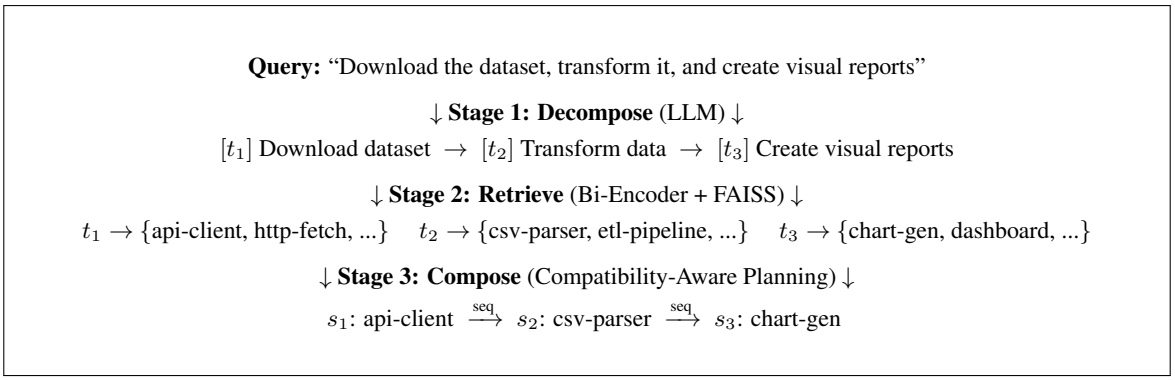

Figure 1: Overview of SKILLWEAVER. A complex query is decomposed into atomic sub-tasks, each matched to candidate skills via bi-encoder retrieval, then composed into an executable skill chain using compatibility-aware planning.

cases where the LLM produces numbered lists or other non-JSON formats.

4.2 Stage 2: Skill Retrieval

For each sub-task t_k, we retrieve the top-m candidate skills from the library using a bi-encoder architecture:

$$\text{cand}(t_k) = \text{top-}m_{s \in \mathcal{S}} \ \cos(E_q(t_k), E_s(s)) \quad (3)$$

where E_q and E_s are the query and skill encoders (shared parameters), implemented using BGE-large-en-v1.5 (Xiao et al., 2023).

Skill Representation. We investigate two encoding strategies:

- **Metadata-only**: $\text{repr}(s) = n_s \oplus d_s$ (name and description)
- **Body-aware**: $\text{repr}(s) = n_s \oplus d_s \oplus b_s[:L]$ (with truncated body)

where $\oplus$ denotes concatenation and $L = 2000$ characters.

Embeddings are L_2-normalized and indexed using FAISS (Johnson et al., 2019) for efficient inner product search.

4.3 Stage 3: Compatibility-Aware Planning

Given candidates for each sub-task, the planner selects the final skill per step while considering sequential compatibility:

$$\sigma(t_k) = \arg \max_{s \in \text{cand}(t_k)} \ \alpha \cdot \text{sim}(t_k, s) + (1 - \alpha) \cdot \text{compat}(s_{k-1}, s) \quad (4)$$

where $\text{compat}(s_a, s_b)$ combines embedding similarity, category overlap, and I/O pattern heuristics. The execution DAG assumes sequential dependencies by default, which we find covers the vast majority of compositional queries in practice.

Category	Count	Examples
Code Generator	389	prd-generator, nextjs-dev
API Client	312	stripe-api, github-api
Test Writer	248	unit-test-gen, qa-suite
Data Processor	221	csv-parser, etl-pipeline
Code Analyzer	187	code-review, lint-checker
Writer	176	blog-writer, doc-gen
Deployment	142	k8s-deploy, ci-cd-setup
Security	118	vuln-scanner, audit-tool
Visualizer	97	chart-gen, dashboard-builder
Search	96	web-search, code-search
+ 7 more categories		609 total

Table 1: Top 10 skill categories in COMPSKILLBENCH (of 17 total). The full pool contains 2,595 skills from 212 repositories.

5 Benchmark: COMPSKILLBENCH

5.1 Skill Pool Construction

We collect skills from 212 public GitHub repositories that adopt the SKILL.md specification format (Anthropic, 2025). The raw collection yields 11,011 skills. We apply the following curation pipeline:

1. **Deduplication**: Skills with identical names are merged, reducing to 7,681 unique skills.
2. **Quality filtering**: Skills without descriptions or with descriptions shorter than 10 characters are removed.
3. **Categorization**: Skills are assigned to 17 functional categories (Table 1) using keyword matching on name and description fields only. Anti-keywords prevent false positive assignments (e.g., "debug" skills are excluded from "deployment"). Skills matching no category or multiple conflicting categories are excluded.
4. The final pool contains **2,595** categorized skills.

5.2 Query Generation

Compositional queries are generated by combining skills from different categories into multi-step tasks:

Difficulty Levels.
- **Easy** (117 queries): 2 skills, 2 categories
- **Medium** (120 queries): 3 skills, 3 categories
- **Hard** (63 queries): 4–5 skills, 4–5 categories

Each query is associated with ground-truth sub-task descriptions (derived from actual skill descriptions), ground-truth skill IDs, required categories, and a sequential execution order. The benchmark totals 300 queries involving 523 unique ground-truth skills.

5.3 Evaluation Metrics

We evaluate at three granularities:

Step-Level Metrics.
- **Skill Recall@k** (R@k): Fraction of steps where the ground-truth skill appears in the top-k candidates.
- **Category Recall@k** (CatR@k): Fraction of steps where *any skill from the correct category* appears in the top-k. This relaxed metric is more practical, as many skills within a category are functionally interchangeable.

Chain-Level Metrics.
- **Chain Exact Match**: Fraction of queries where *all* steps select the exact ground-truth skill.
- **Chain Category Match** ($\text{Chain}_{\text{cat}}$): Average fraction of steps per query that select a skill from the correct category.

Decomposition Accuracy. Fraction of queries where the predicted number of sub-tasks matches the ground truth.

6 Experimental Setup

LLM Decomposers. We compare three open-weight 7–8B instruction-tuned models: Qwen2.5-7B-Instruct (Qwen Team, 2024), Llama-3.1-8B-Instruct (Dubey et al., 2024), and Mistral-7B-Instruct-v0.3 (Jiang et al., 2023). Generation uses temperature $\tau = 0.1$, $\text{top}_p = 0.9$, and a maximum of 256 new tokens.

Retriever. BGE-large-en-v1.5 (Xiao et al., 2023) (1024-dim) serves as the bi-encoder, with FAISS IndexFlatIP for exact inner product search. We set $k = 10$ for retrieval unless otherwise noted.

Baselines.
- **Single-Skill**: No decomposition; the full query is used to retrieve one skill set applied to all steps. This represents current single-skill routing methods.
- **Oracle Decomposition**: Ground-truth sub-task descriptions are used for retrieval, establishing an upper bound on decomposition quality.

Hardware. All experiments run on a single NVIDIA A100-80GB GPU with 491GB system RAM.

7 Results

7.1 Main Results

Table 2 presents the main experimental results across all configurations.

Decomposition is the bottleneck. The most striking result is the enormous gap between oracle decomposition and LLM decomposition. Oracle decomposition achieves R@1 of 99.5% and perfect category accuracy, demonstrating that the retriever is highly effective when given accurate sub-task descriptions. The best LLM decomposer (Qwen2.5-7B) reaches only CatR@1 = 36.3%, indicating that decomposition quality—not retrieval capability—is the primary bottleneck.

Compositional routing outperforms single-skill. Despite imperfect decomposition, our full pipeline (Qwen + meta) achieves CatR@1 = 33.9%, substantially outperforming the single-skill baseline at 21.1%. This validates that even approximate decomposition provides value for compositional routing.

Metadata matches or outperforms body. For oracle decomposition, metadata-only retrieval achieves R@1 = 99.5% versus 99.0% for body-aware retrieval. In the full pipeline, body-aware has a slight edge at the category level (CatR@1: 36.3% vs 33.9%), but the difference is small and comes with significantly higher encoding cost due to longer input sequences. This suggests that concise skill metadata (name + description) captures the essential discriminative signal, consistent with the intuition that good skill descriptions should be self-contained summaries.

7.2 Decomposer Comparison

Table 3 compares the three LLM decomposers. Qwen2.5-7B significantly outperforms the other

Method	Signal	Decomp%	R@1	R@10	CatR@1	CatR@10	Chain_E	$\text{Chain}_{\text{cat}}$
Baselines								
Single-Skill	Meta	–	0.000	0.029	0.211	0.698	0.000	0.216
Single-Skill	Body	–	0.002	0.028	0.285	0.633	0.000	0.293
Oracle Decomp	Meta	1.000	**0.995**	**1.000**	**1.000**	**1.000**	**0.987**	**1.000**
Oracle Decomp	Body	1.000	0.990	1.000	0.998	1.000	0.970	0.998
*Full Pipeline (*SKILLWEAVER*)*								
Qwen2.5-7B	Meta	0.360	0.005	0.034	0.339	0.649	0.000	0.316
Qwen2.5-7B	Body	0.370	0.008	0.029	<u>0.363</u>	<u>0.662</u>	0.000	<u>0.345</u>
Llama-3.1-8B	Meta	0.000	0.002	0.016	0.216	0.507	0.000	0.210
Mistral-7B	Meta	0.177	0.005	0.036	0.243	0.607	0.000	0.248

Table 2: Main results on COMPSKILLBENCH. **Bold**: best overall. <u>Underline</u>: best among full pipeline configurations. R@k: Skill Recall@k. CatR@k: Category Recall@k. Chain_E: Chain Exact Match. $\text{Chain}_{\text{cat}}$: Chain Category Match. Decomp%: decomposition accuracy (correct number of sub-tasks).

Decomposer	Decomp%	CatR@1	$\text{Chain}_{\text{cat}}$
Qwen2.5-7B	**0.360**	**0.339**	**0.316**
Mistral-7B	0.177	0.243	0.248
Llama-3.1-8B	0.000	0.216	0.210
Oracle	1.000	1.000	1.000

Table 3: Decomposer comparison (all using metadata-only retrieval). Qwen2.5-7B achieves the best decomposition accuracy and downstream routing performance.

models, achieving 36.0% decomposition accuracy versus 17.7% for Mistral and 0% for Llama.

Over-decomposition is the primary failure mode. Analysis of decomposition patterns (Figure 2) reveals that Llama-3.1-8B systematically over-decomposes: its median output is 9 sub-tasks, while the ground truth distribution peaks at 2–3. No single Llama decomposition matches the ground-truth count, explaining its 0% decomposition accuracy. Mistral shows a similar but less severe tendency, while Qwen's distribution most closely aligns with the ground truth.

7.3 Difficulty Analysis

Table 4 breaks down results by difficulty level. A notable finding is that the decomposition-based pipeline shows its *greatest advantage on harder queries*: for hard queries (4–5 skills), Qwen + Meta achieves CatR@1 = 41.0% versus the single-skill baseline at 14.6%, a relative improvement of 181%. For easy queries (2 skills), the improvement is more modest (21.4% vs 20.1%). This pattern suggests that decomposition becomes increasingly important as task complexity grows.

Method	Diff.	CatR@1	CatR@10
Single-Skill	Easy	0.201	0.774
	Medium	0.267	0.667
	Hard	0.146	0.675
Qwen + Meta	Easy	0.214	0.637
	Medium	0.367	0.658
	Hard	**0.410**	0.646
Qwen + Body	Easy	0.252	0.607
	Medium	**0.403**	**0.728**
	Hard	0.407	0.623

Table 4: Performance by difficulty level. Decomposition-based routing shows increasing advantage on harder queries.

7.4 Ablation: Constrained Decomposition

To isolate the effect of decomposition *granularity* (number of steps) from decomposition *quality* (sub-task descriptions), we run an ablation where LLM decompositions are truncated or padded to match the ground-truth step count.

The constrained decomposition achieves CatR@1 = 33.9%, identical to the unconstrained pipeline. This indicates that the primary challenge is not predicting the *number* of steps but generating sub-task descriptions that are *semantically aligned* with the target skills. Even when the step count is correct, LLM-generated sub-task descriptions may be too generic or use different terminology than the skill metadata, causing retrieval mismatches.

8 Analysis

8.1 Error Analysis

We manually examine 50 randomly selected failure cases from the Qwen + Meta pipeline to cate-

gorize error types:

- **Over-decomposition** (36%): The LLM splits a query into more steps than necessary, inserting intermediate steps not in the ground truth (e.g., "identify necessary patches" inserted between "run security audit" and "apply patches").
- **Generic sub-task descriptions** (28%): The LLM produces descriptions that are too broad (e.g., "process the data" instead of "transform CSV data using pandas"), leading to retrieval of tangentially related skills.
- **Vocabulary mismatch** (22%): The sub-task description uses different terminology than the skill metadata (e.g., "create visual reports" vs. skill named "matplotlib-chart-generator").
- **Under-decomposition** (14%): The LLM merges multiple steps into one, typically for hard queries with 4+ required skills.

8.2 Case Study

Successful routing (Q0): "*Generate the application code, write tests, and deploy to production*" is decomposed into three sub-tasks matching the ground truth exactly. The retriever correctly identifies a code generator, a QA tool, and a deployment orchestrator.

Partial success (Q100): "*Search for sources, process findings, visualize trends, write report, and export document*" is correctly decomposed into 5 steps. The retriever correctly identifies skills for search, data processing, and report writing (3/5 categories correct), but selects an exploratory data analysis skill instead of a dedicated visualizer.

Over-decomposition failure (Q200): "*Draft a document and localize it for international readers*" requires 2 skills (writer + translator), but the LLM produces 7 sub-tasks including review, translation, proofreading, and cultural adaptation steps. The first step correctly selects a document writing skill, but subsequent steps diverge from the ground truth.

8.3 Retrieval Quality with Oracle Decomposition

The near-perfect oracle performance (R@1 = 99.5%) deserves further examination. The 0.5% of oracle failures occur when multiple skills in the library have very similar metadata, making exact ID matching ambiguous even though the correct *category* of skill is retrieved (category recall remains 100%). This validates our design choice of including category-level metrics, which better reflect practical routing quality.

9 Discussion

The decomposition gap. Our results quantify a "decomposition gap": the difference between what is achievable with perfect sub-task descriptions (oracle) and what current LLMs produce. At CatR@1, this gap is 66 percentage points (100% – 33.9%). Closing this gap is the most impactful direction for improving compositional skill routing. Potential approaches include fine-tuning decomposers on skill-aware data, iterative refinement with retrieval feedback, and leveraging skill metadata as decomposition guidance.

Metadata vs. body. Our finding that metadata-only retrieval is competitive with body-aware retrieval has practical implications: it means that skill authors should invest in writing clear, descriptive metadata (names and descriptions) rather than relying on the full specification body to be the primary retrieval signal. This finding contrasts with Xu et al. (2025), who report that body content is the "decisive" signal for single-skill routing. We hypothesize that the compositional setting, where queries are decomposed into specific sub-tasks, produces more targeted retrieval queries that align naturally with concise metadata.

Scalability. Our FAISS-based retrieval scales to thousands of skills with sub-second latency per query. The primary computational cost is LLM-based decomposition, which at approximately 0.8 seconds per query (sequential generation on A100) is practical for interactive use. Batch processing reduces amortized cost further.

Limitations. Our benchmark uses template-based query generation, which may not fully capture the diversity of real-world compositional queries. The sequential DAG assumption, while covering most cases in our benchmark, does not model parallel skill execution. We evaluate with 7–8B parameter LLMs; larger models may achieve better decomposition quality. Finally, our evaluation is retrieval-focused and does not measure end-to-end task completion with actual skill execution.

10 Conclusion

We have formalized the Compositional Skill Routing problem and presented SKILLWEAVER, a

decompose-retrieve-compose framework for routing complex queries to multiple skills. Through CompSkillBench, a new benchmark of 300 compositional queries over 2,595 curated skills, we demonstrate that task decomposition quality is the primary bottleneck in compositional routing: oracle decomposition achieves near-perfect retrieval, while the best 7B LLM decomposer reaches 33.9% category recall. Our analysis identifies over-decomposition and vocabulary mismatch as the dominant failure modes, providing clear directions for future work on decomposition-aware skill routing.

References

Anthropic. 2025. Agent skills specification. Https://docs.anthropic.com/en/docs/agents-and-tools/agent-skills.

Abhimanyu Dubey and 1 others. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.

Shibo Hao, Tianyang Liu, Zhen Wang, and Zhiting Hu. 2024. Toolkengpt: Augmenting frozen language models with massive tools via tool embeddings. *Advances in Neural Information Processing Systems*, 36.

Wenlong Huang, Pieter Abbeel, Deepak Pathak, and Igor Mordatch. 2022. Language models as zero-shot planners: Extracting actionable knowledge for embodied agents. *Proceedings of the 39th International Conference on Machine Learning*.

Albert Q Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, and 1 others. 2023. Mistral 7b. *arXiv preprint arXiv:2310.06825*.

Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2019. Billion-scale similarity search with gpus. *IEEE Transactions on Big Data*, 7(3):535–547.

Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense passage retrieval for open-domain question answering. *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*.

Jianmo Ni, Gustavo Hernandez Abrego, Noah Constant, Ji Ma, Keith B Hall, Daniel Cer, and Yinfei Yang. 2022. Sentence-t5: Scalable sentence encoders from pre-trained text-to-text models. *Findings of the Association for Computational Linguistics: ACL 2022*.

Shishir G Patil, Tianjun Zhang, Xin Wang, and Joseph E Gonzalez. 2024. Gorilla: Large language model connected with massive apis. In *Proceedings of the 41st International Conference on Machine Learning*.

Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, and 1 others. 2023. Toolllm: Facilitating large language models to master 16000+ real-world apis. *arXiv preprint arXiv:2307.16789*.

Qwen Team. 2024. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 36.

Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. 2023. Taskbench: Benchmarking large language models for task automation. *arXiv preprint arXiv:2311.18760*.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36.

Lei Wang, Wanyu Xu, Yihuai Lan, Zhiqiang Hu, Yunshi Lan, Roy Ka-Wei Lee, and Ee-Peng Lim. 2023. Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models. *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc V Le, and Denny Zhou. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35.

Shitao Xiao, Zheng Liu, Peitian Zhang, and Niklas Muennighoff. 2023. C-pack: Packaged resources to advance general chinese embedding. *arXiv preprint arXiv:2309.07597*.

Qinyuan Xu and 1 others. 2025. Skillrouter: Adaptive skill-based routing for agentic systems. *arXiv preprint arXiv:2603.22455*.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. React: Synergizing reasoning and acting in language models. *Proceedings of the International Conference on Learning Representations*.

Denny Zhou, Nathanael Schärli, Le Hou, Jason Wei, Nathan Scales, Xuezhi Wang, Dale Schuurmans, Claire Cui, Olivier Bousquet, Quoc Le, and Ed Chi. 2022. Least-to-most prompting enables complex reasoning in large language models. *arXiv preprint arXiv:2205.10625*.

A Category Taxonomy

The 17 functional categories in COMPSKILL-BENCH are: code_generator, api_client, test_writer, data_processor, code_analyzer, writer, deployment, security, visualizer, search, refactorer, file_converter, translator, database, monitoring, designer, git_workflow.

B Decomposition Distribution

Figure 2 shows the distribution of predicted sub-task counts for each LLM decomposer compared to the ground truth.

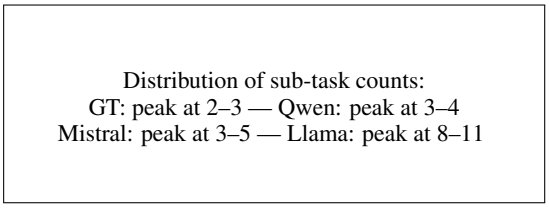

Figure 2: Distribution of predicted sub-task counts by decomposer. Llama-3.1-8B systematically over-decomposes, with a median of 9 sub-tasks versus 3 in the ground truth.

C Top-k Sensitivity

With oracle decomposition, R@1 = 99.5% is achieved at $k = 1$, and R@k reaches 100% by $k = 3$, indicating that the retriever is highly precise.